# Haskell 期中速查表

单面 A4 · 第 1-13 讲精选

独立模板与等价式；实例方法省略 instance 外壳，按相应类型使用。保留难记结构，删去基础语法与普通函数签名。

## 01 Fold：展开、改写、结构递归

第 3、8、9 讲

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldl :: (b -> a -> b) -> b -> [a] -> b
foldr f z (x:xs) = f x (foldr f z xs)
foldl f z (x:xs) = foldl f (f z x) xs
foldr f z [a,b,c] = f a (f b (f c z))
foldl f z [a,b,c] = f (f (f z a) b) c
map g = foldr (\x ys -> g x:ys) []
filter p = foldr (\x ys ->
                  if p x then x:ys else ys) []
reverse = foldl (flip (:)) []
length  = foldr (\_ n -> n+1) 0
xs ++ ys = foldr (:) ys xs
foldr f z (map g xs) = foldr (f . g) z xs
foldN h z 0 = z                -- n >= 0
foldN h z n = h (foldN h z (n-1))
```

两种列表 fold 在 [] 上均返回 z；方向决定括号，f 不必结合。foldr 可短路，foldl 必须到表尾；严格累积用 Data.List.foldl'。

## 02 抽象操作与 Laws

第 12 讲

Monad 要求 Applicative，后者要求 Functor。以下操作签名省略各自的类约束；<$> = fmap，return = pure。

```
fmap  :: (a -> b) -> f a -> f b
pure  :: a -> f a
(<*>) :: f (a -> b) -> f a -> f b
(>>=) :: m a -> (a -> m b) -> m b
do { x <- u; k x } = u >>= \x -> k x
u >> v = u >>= \_ -> v
(f >=> g) x = f x >>= g
-- Functor
fmap id = id
fmap (f . g) = fmap f . fmap g
-- Applicative
pure id <*> u = u
pure f <*> pure x = pure (f x)
u <*> pure y = pure ($ y) <*> u
pure (.) <*> u <*> v <*> w = u <*> (v <*> w)
-- Monad
pure x >>= f = f x
u >>= pure = u
(u >>= f) >>= g = u >>= (\x -> f x >>= g)
```

```
-- 函数实例 ((->) r)：共享同一环境
fmap = (.)                          -- pure=const
(f <*> g) r = f r (g r)
(h >>= k) r = k (h r) r
```

```
-- k :: a -> m b：fmap k mx :: m (m b)
mx >>= k = join (fmap k mx)
join u = u >>= id
```

## 03 自定义数据：结构递归与 fold

第 9 讲

每个构造器一条分支，递归字段先递归；以函数替换构造器。下面 size 计算 Val 叶子数。

```
data Expr = Val Int | Add Expr Expr | Mul Expr Expr
folde :: (Int -> a) -> (a -> a -> a)
      -> (a -> a -> a) -> Expr -> a
folde v a m (Val n) = v n
folde v a m (Add l r) = a (folde v a m l) (folde v a m r)
folde v a m (Mul l r) = m (folde v a m l) (folde v a m r)
eval = folde id (+) (*)
size = folde (const 1) (+) (+)
```

## 04 Tree Functor 与 State 重标注

第 12 讲

```
data Tree a = Leaf a | Node (Tree a) (Tree a)
fmap f (Leaf x) = Leaf (f x)
fmap f (Node l r) = Node (fmap f l) (fmap f r)
alabel (Leaf _) = Leaf <$> fresh
alabel (Node l r) = Node <$> alabel l <*> alabel r
relabel t = fst (app (alabel t) 0)
```

fresh / app / ST 见右栏；从左到右给叶子编号。若数据存在非叶节点：fmap 要递归左右子树，同时映射当前节点的值。

## 05 Maybe / List / State

第 12 讲

```
-- Maybe：pure = Just；Nothing 传播失败
Just x >>= k = k x
Nothing >>= _ = Nothing
-- []：所有组合（不是按位置配对）
fs <*> xs = [f x | f <- fs, x <- xs]
xs >>= k = concatMap k xs          -- pure x=[x]
-- State：返回顺序为（结果，新状态）
newtype StateTrans a = ST (Int -> (a,Int))
app (ST f) = f
pure x = ST $ \s -> (x,s)
fmap g st = ST $ \s -> let (x,s') = app st s
                       in (g x,s')
st >>= k = ST $ \s -> let (x,s') = app st s
                      in app (k x) s'
fresh = ST $ \n -> (n,n+1)
u <*> v = ST $ \s ->
  let (f,s')  = app u s
      (x,s'') = app v s'
  in (f x,s'')
```

## 06 课程 Parser：0 或 1 个结果

第 13 讲

```
newtype Parser a = P (String -> [(a,String)])
app (P f) = f
fmap g p = P $ \s ->
  [(g x,out) | (x,out) <- app p s]
pure x = P $ \s -> [(x,s)]
p >>= k = P $ \s -> case app p s of
  [] -> []; [(x,out)] -> app (k x) out
empty = P $ \_ -> []
p <|> q = P $ \s -> case app p s of
  [] -> app q s; rs -> rs
item = P $ \s -> case s of
  [] -> []; (x:xs) -> [(x,xs)]
sat p = do { x <- item;
             if p x then pure x else empty }
char c = sat (== c)
string [] = pure []
string (x:xs) = (:) <$> char x <*> string xs
many p = some p <|> pure []
some p = (:) <$> p <*> many p
nat = read <$> some (sat isDigit)
space = many (sat isSpace) >> pure ()
token p = do { space; x <- p; space; pure x }
symbol s = token (string s)
expr = do { t <- term;
  (do { symbol "+"; e <- expr; pure (t+e) })
    <|> pure t }
term = do { f <- factor;
  (do { symbol "*"; t <- term; pure (f*t) })
    <|> pure f }
factor = (do { symbol "("; e <- expr;
               symbol ")"; pure e }) <|> token nat
```

[] 失败，[(v,out)] 成功；完整解析要求 out=""。<|> 左失败后从原输入重试。many 允许 0 次，some 至少 1 次；被重复的 parser 成功时必须消耗输入。右递归使运算右结合，* 高于 +。

```
-- 对 Monad 可用此式补出 <*> (Parser 也适用)
u <*> v = u >>= \f -> v >>= \x -> pure (f x)
```

## 07 汇集效果：第 12 讲的列表版本

第 11-12 讲

```
sequenceA :: Applicative f => [f a] -> f [a]
sequenceA [] = pure []
sequenceA (u:us) = (:) <$> u <*> sequenceA us
sequence :: Monad m => [m a] -> m [a]
sequence [] = pure []
sequence (u:us) = do { x <- u; xs <- sequence us;
                       pure (x:xs) }
forM xs f = sequence (map f xs)
when p u = if p then u else pure ()
```

Maybe 任一失败则整体失败；[] 枚举所有组合；IO 依次执行并收集结果。return 只包装值，不提前退出；do 内 let 定义纯值，无 in。